States of the Exporter

During the simulation runs *Plant Simulation* shows the **state of the Exporter** as one or more **horizontally arranged cubes** along the top of the picture of the object in the *Frame*.

Remarks

These states are designated by different colors.

Description	Color of the State Graphic
The *Exporter* is **failed**.	red
The *Exporter* **exports** services.	green
The *Exporter* is **paused**.	blue

See also

Tab States > **State Graphic [defined]** in 3D

States of the Material Flow Objects

States of the Energy Features

States of the Frame

States of the Transporter

States of the Method

States of the MQTT Interface

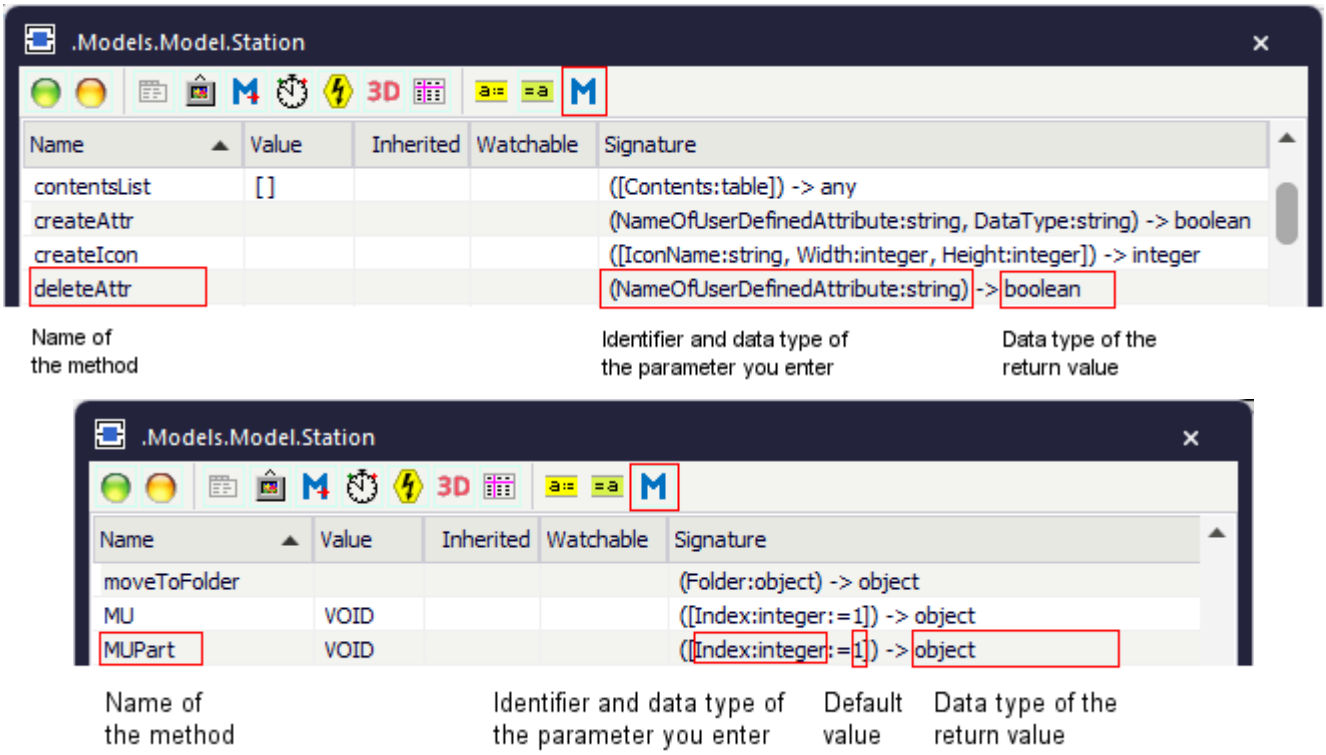
Methods of the Exporter

_Methods of the Exporter

The *Exporter* provides:

- The *methods* listed in the table of contents to the left.
- The **Methods of All Objects**.

To view all of the *methods*, *read-only attributes*, and *attributes* of the object, open the window **Show Attributes and Methods**. The figure below illustrates the information using the example of the object *Station*.



- Select **Show Attributes and Methods** on the context menu of the **Class Library** to show the *methods*, *read-only attributes*, and *attributes* of the selected **Class [general description]**.
- Press the **F8** key or click **Show Attributes and Methods** on the **Home** ribbon tab of the *Frame* into which you inserted an instance to show the *methods*, *read-only attributes*, and *attributes* of the selected **Instance [general description]**.

An example of the **Syntax** line of the individual methods might look like this:

```
<Path>.openDialog([CallOpenControl:boolean:=false]) → boolean
```

- The expression *<Path>* designates the path of the object to which the *method* applies.
- The signature of the method, consisting of the identifier and the data type of the parameter, is listed in parentheses. The expression *(Parameter:string)*, for example, designates a parameter of data type *string*. Instead of a constant value, you can also use a variable of the required type or a *method* that returns the required data type.

> **Note**
>
> Make sure to enter the parentheses for expressions within parentheses (...). Not entering them may lead to unexpected results and open the *Debugger*.

Optional parameters are listed within brackets. The expression *[,Parameter:boolean]*, for example, means that you can, but do not have to enter the *boolean* parameter.

If a parameter has a default value, the signature shows the default value after the parameter, `:= false` in the example above.

If the *method* has a return value, the signature shows its data type after the arrow ->, → `boolean` in the example above.

expStat [SimTalk]

Returns the statistics table of the *Exporter* designated by <Path> and writes it into a table.

Type

Method

Syntax

```
<Path>.expStat → boolean
<Path>.expStat(StatisticsTable:table) → boolean
```

Parameter

The parameter *StatisticsTable* of data type *table* designates the name of the table.

Return Value

The return value has the data type *boolean*.

true if the call was successful.

false if the object does not collect any statistics data.

Example

```
var MyExporterStatisticsTable: table
MyExporter.expStat(MyExporterStatisticsTable)
```

findNewImporter [SimTalk]

Registers the *Exporter* designated by <Path> as being available with its *Broker*.

Remarks

The *Broker* then searches the list of all open requests and attempts to find new *Exporters*. The fact that the *Exporter* is available to fulfill new orders is passed along to all *Brokers* connected by *Connectors*.

Type

Method

Syntax

```
<Path>.findNewImporter
```

Example

```
MyExporter.findNewImporter
```

See also

Importers [Exporter]

Importers [Worker]

getExportedServices [SimTalk]

Returns all services which the *Exporter/Worker* designated by <Path> exports at the moment.

Type

Method

Syntax

```
<Path>.getExportedServices([Services:table]) -> any
```

Parameter

The optional parameter *Services* of data type *table* designates the name of the table into which *Plant Simulation* writes the services.

The table has one column. It contains all services that the *Exporter/Worker* exports at the moment. The subtable contains all *importers* importing that service and the number of services which the *Exporter* provides for the *importer* (*integer*).

Return Value

The return value is an *array* containing the *services* which the *Exporter/Worker* currently exports if you do not specify the optional parameter.

Example

```
MyExporter.getExportedServices(MyExportedServicesTable)
MyExporter.getExportedServices         // returns an array
print MyExporter.getExportedServices   // might return [Job1]
```

See also

Services [SimTalk] - Exporter

Services [Exporter]

Exported Services [Exporter]

Exported Services [Worker]

getImporters [SimTalk]

Returns all *importers* for which the *Exporter/Worker* designated by <Path> is fulfilling orders.

Type

Method

Syntax

```
<Path>.getImporters([Importers:table]) -> any
```

Parameter

The optional parameter *Importers* of data type *table* designates the name of the table into which *Plant Simulation* writes the importers.

It has two columns:

- Column 1 of data type *object* contains all *importers* for which the *Exporter* is fulfilling orders at the moment.
- Column 2 of data type *integer* designates its type: `0` designates the *failure/remove failure-importer*, `1` the *set-up-importer*, `2` the*processing-importer*, and `3` the *transport-importer*.

If you do not specify the optional parameter, *Plant Simulation* returns an *array* with the assigned *objects* which currently import the *Exporter/Worker*.

Return Value

The return value has the data type *any*.

Example

```
MyExporter.getImporters(MyImportersTable)
MyExporter.getImporters         // returns an array
print MyExporter.getImporters   // might return
[*.Models.MyProcessingAndSetupBroker.Station3]
```

See also

Importers [Exporter]

Importers [Worker]

hasService [SimTalk]

Returns if the *Exporter/Worker* designated by <Path> supports the designated service (true) or not (false).

Type

Method

Syntax

```
<Path>.hasService(ServiceName:string) → boolean
```

Parameter

The parameter *ServiceName* of data type *string* designates the name of the service.

Return Value

The return value has the data type *boolean*.

Example

```
MyExporter.hasService("drill")
```

SimTalk

Services [SimTalk] - Exporter

See also

Services [Exporter]

Exported Services [Exporter]

Exported Services [Worker]

Read-Only Attributes of the Exporter

_Read-Only Attributes of the Exporter

The *Exporter* provides:

- The *read-only attributes* listed in the table of contents to the left.
- The **_Read-Only Attributes of All Objects**.